Requirements: Customer Outage Request (Illustrative Case Study)

This document turns the scenario into written requirements and links each one to the BPMN process, the ERD, the OpenAPI spec and the tests of the mock service. All names, numbers and data are illustrative.

1. Purpose and scope

Purpose: restore a customer's internet service by handling an outage request from report to confirmed repair.

In scope: service request intake, contract and SLA check, creation of work (an *activity*) in a team queue (*bucket*), assignment to a qualified technician, escalation, repair and follow-up visit, customer confirmation, activity status tracking.

Out of scope: billing, invoicing, inventory management of spare parts, technician route optimization, the technician matching algorithm (treated as an internal rule), real JWT validation.

2. Stakeholders and actors

Role	Interest
Customer	Gets the outage fixed quickly and can confirm the result
Dispatcher	Keeps work moving and handles exceptions manually
Technician	Receives work they are qualified for and records progress
DEMO Service (system)	Stores activities, applies business rules, exposes the API
Service manager	Needs SLA compliance and a clear audit of what happened

3. Business requirements

ID	Requirement
BR-1	Every valid outage request is resolved within the contracted SLA.
BR-2	Work is only assigned to technicians who hold the skill the work requires.
BR-3	A valid request is never rejected only because no technician is immediately available.
BR-4	The status of every request can be traced from report to completion.

4. Business rules

ID	Rule
RULE-1	A request is valid only if the customer's contract and SLA are valid.
RULE-2	Each activity type requires a work skill (repair requires INTERNET_REPAIR; installation requires CABLE_INSTALL).
RULE-3	A technician can only be assigned an activity whose required skill they hold. A bucket accepts any activity.
RULE-4	If no technician is found within 30 minutes, the work is escalated to a dispatcher.
RULE-5	A resource is either a bucket or a technician.

5. User stories and acceptance criteria

US-1 Report a service problem (Customer)

As a customer, I want to report an internet outage so that the provider can start a repair.

- Given I have a contract, when I submit a problem report, then the provider receives a service request.

US-2 Check contract and SLA (Dispatcher)

As a dispatcher, I want each request checked against the contract and SLA so that only valid requests consume capacity.

- Given a service request, when the contract and SLA are valid, then the process continues.
- Given a service request, when the contract or SLA is not valid, then the request is rejected and the customer is informed.

US-3 Create work in a team queue (System)

As the system, I want to create an activity in a bucket so that work is recorded before a technician is chosen.

- Given a valid request, when the activity is created, then it is stored with status `pending`, an `activityId`, and the required work skill derived from the activity type (response 201).
- Given an invalid body or unknown activity type, then the response is 400 and nothing is stored.
- Given an unknown resource, then the response is 404 and nothing is stored.

US-4 Find a qualified technician (System)

As the system, I want to look for a technician by skill, zone and availability so that work goes to someone who can do it.

- Given an activity in a bucket, when a qualified technician is found within 30 minutes, then the activity is reassigned to that technician.

US-5 Escalate unmatched work (Dispatcher)

As a dispatcher, I want unmatched work escalated to me so that valid requests are not lost.

- Given no technician is found within 30 minutes, then the work is escalated to a dispatcher for manual assignment and the request is not rejected.

US-6 Reassign with skill validation (System)

As the system, I want to validate the technician's skill on reassignment so that unqualified assignments are impossible.

- Given an activity and a technician who holds the required skill, when the activity is reassigned, then the assignment changes (response 200).
- Given a technician who lacks the required skill, then the response is 422, the activity is unchanged, and the process routes to manual handling.

US-7 Perform repair and follow-up (Technician)

As a technician, I want to record progress so that the customer and dispatcher see the real status.

- Given an assigned activity, when I start work, then the status becomes `started`.
- Given a spare part is missing, then a follow-up visit is scheduled and the repair step is repeated.
- Given the repair is done, then the customer is notified that the repair is completed.

US-8 Confirm completion (Customer)

As a customer, I want to confirm the repair so that the request can be closed.

- Given the repair is completed, when I confirm, then the activity status becomes `completed` and the request is resolved.

6. Functional requirements

Priority uses MoSCoW (Must, Should, Could).

ID	Requirement	Priority	Story
FR-1	The API creates an activity (`POST /activities`) from `activityType`, `date` and `resourceId`, with initial status `pending`.	Must	US-3
FR-2	The system derives `workSkill` from `activityType`; clients cannot set it.	Must	US-3
FR-3	The API validates required fields and allowed activity types and returns 400 otherwise.	Must	US-3
FR-4	The API returns 404 when the referenced resource or activity does not exist.	Must	US-3, US-6
FR-5	The API reassigns an activity by updating `resourceId` (`PATCH /activities/{id}`).	Must	US-6

ID	Requirement	Priority	Story
FR-6	The API returns 422 when a technician lacks the required skill and leaves the activity unchanged; buckets accept any activity.	Must	US-6
FR-7	The API updates status through `pending`, `started`, `completed`.	Must	US-7, US-8
FR-8	The API returns an activity by ID (`GET /activities/{id}`).	Should	US-4, US-7
FR-9	The API rejects an empty `PATCH` body with 400.	Should	US-6, US-7
FR-10	The process escalates to a dispatcher when no technician is found within 30 minutes.	Must	US-5
FR-11	The customer is informed when a request is rejected and when a repair is completed.	Should	US-2, US-7
FR-12	A follow-up visit can be scheduled when a spare part is missing.	Could	US-7
FR-13	All API endpoints require a bearer token; a missing or invalid token returns 401.	Must	all

7. Non-functional requirements

Targets are illustrative and would be confirmed with stakeholders.

ID	Category	Requirement	Priority
NFR-1	Performance	95% of API requests are answered in under 1 second.	Should
NFR-2	Security	Authentication uses a bearer token on every endpoint; production use requires JWT validation.	Must
NFR-3	Data integrity	A failed reassignment leaves the activity unchanged (no partial updates).	Must
NFR-4	Auditability	The system records who changed an assignment or status, and when.	Should
NFR-5	Reliability	The API is available at least 99.5% of business hours.	Should
NFR-6	Consistency	Errors use one response format (`{"message": "..."}`).	Should

8. Data requirements

Conceptual model: see the ERD.

Entity	Key attributes	Notes
Activity	activityId, activityType, date, status, resourceId, workSkill	Created in a bucket, then reassigned
Resource	resourceId, name, type (bucket or technician)	Has many work skills
Work Skill	name	Many-to-many with Resource and with Activity
Service Request, Route, Inventory, User	conceptual only	Not implemented in the mock service

9. Interface requirements (API summary)

Method and path	Purpose	Responses
`POST /activities`	Create an activity	201, 400, 401, 404, 422
`GET /activities/{activityId}`	Read an activity	200, 401, 404
`PATCH /activities/{activityId}`	Reassign or change status	200, 400, 401, 404, 422

10. Traceability

BPMN step	Story	Requirements	API	Entity	Verified by (smoke test)
Report problem, request received	US-1	n/a	out of API scope	Service Request	not covered
Check contract / SLA, reject	US-2	RULE-1, FR-11	internal rule	Service Request	not covered
Create activity in the bucket	US-3	FR-1 to FR-4, FR-13	`POST /activities`	Activity, Resource	201 create; 400 invalid body; 404 unknown resource; 401 without token
Find technician, 30-minute timer	US-4, US-5	RULE-4, FR-10	internal matching	Resource, Work Skill	not covered (escalation modeled in BPMN)
Reassign activity to technician	US-6	FR-5, FR-6, NFR-3	`PATCH /activities/{id}`	Activity, Resource, Work Skill	200 qualified technician; 422 without skill
Perform repair, follow-up visit	US-7	FR-7, FR-12	PATCH (status)	Activity	200 status started
Confirm completion	US-8	FR-7	PATCH (status completed)	Activity	200 status completed

11. Assumptions, constraints and open questions

Assumptions

- Matching by zone and availability is an internal rule and is not specified here.
- Activity types and their required skills are fixed in this demo.

Open questions

- How exactly is the SLA defined: time to respond, or time to restore service?
- What does the dispatcher decide after escalation, and how is that recorded? In the demo, the manual task does not assign a technician (known simplification).
- Should an activity be creatable directly on a technician? The mock applies the skill rule and can return 422, so the OpenAPI spec needs a 422 response on `POST`.
- Spare-part inventory is only conceptual; no requirements cover stock or ordering.